

Block 4 — the Hubbard atom from partial spectral functions

Numerical companion to

F. B. Kugler, S.-S. B. Lee and J. von Delft, *Multipoint Correlation Functions: Spectral Representation and Numerical Evaluation*, [Phys. Rev. X **11**, 041006 \(2021\)](#)

The plan is to assemble the Matsubara correlators of the half-filled Hubbard atom directly from their spectral representation, and to check them against the closed forms the paper quotes:

Step	Content	Benchmark
1	spectrum: $H, d_\sigma, d_\sigma^\dagger, n_\sigma$; diagonalise; $\rho_i = e^{-\beta E_i} / Z$	$Z = 2(1 + e^{\beta u})$ by hand
2	partial spectral functions, Eq. (28), general in ℓ	—
3	the kernel, Eq. (46), two-branch form	finite by construction
4	$\ell = 2$: assemble Eq. (39)	$G(i\nu) = \frac{1}{2} \sum_{\pm} (i\nu \pm U/2)^{-1}$, App. E
5	$\ell = 4$: 24 permutations, subtract Eq. (73), amputate Eq. (76)	Eqs. (85a), (85b)
6	null test at $U = 0$	every connected object vanishes

All six steps are implemented and checked. The test suite

(`julia --project test/runtests.jl`) carries 46,279 assertions.

Model: $H = \varepsilon_d(n_\uparrow + n_\downarrow) + Un_\uparrow n_\downarrow$ at $\varepsilon_d = -U/2$, on the four states $|0\rangle, |\uparrow\rangle, |\downarrow\rangle, |\uparrow\downarrow\rangle$. This is the $U/\Delta \rightarrow \infty$ limit of the symmetric AIM, Eq. (79).

```
In [1]: include(joinpath(@__DIR__, "..", "src", "HubbardAtom.jl"))
using .HubbardAtom
using LinearAlgebra
using Printf
```

Step 1 — spectrum

Build the operators with correct fermionic signs, diagonalise H , and form the Boltzmann weights. Everything later hangs off the eigenbasis produced here: Eq. (28) needs the matrix elements $(O)_{i,i+1}$ in the energy eigenbasis, which `to_eigenbasis` supplies.

The operators come from a Jordan–Wigner transform with fermionic mode order $(\uparrow, \downarrow)$,

$$d_{\uparrow} = 1 \otimes f, \quad d_{\downarrow} = f \otimes z, \quad f = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

so the parity string z over the $\uparrow$ mode is what produces $d_{\downarrow}|\uparrow\downarrow\rangle = -|\uparrow\rangle$. That sign is not cosmetic: it feeds every permutation sign ζ_p in Eq. (39).

In [2]:

```
U, β = 2.0, 5.0
model = HubbardAtomModel(U, β)
ops = operators(model)

@printf("U = %.3f, β = %.3f, εd = %.3f, u = U/2 = %.3f\n",
        model.U, model.β, level_position(model), half_interaction(model))
println("\nFock basis order: ", join(BASIS_LABELS, " "))
println("\nH (Fock basis) =")
display(ops.H)
```

U = 2.000, β = 5.000, εd = -1.000, u = U/2 = 1.000

Fock basis order:
|0⟩ |↑⟩ |↓⟩ |↑↓⟩

H (Fock basis) =
4x4 Matrix{Float64}:
0.0 0.0 0.0 0.0
0.0 -1.0 0.0 0.0
0.0 0.0 -1.0 0.0
0.0 0.0 0.0 0.0

In [3]:

```
# Fermionic algebra: the checks that catch a wrong Jordan-Wigner sign.
I4 = Matrix{Float64}(I, DIM, DIM)

println("{d↑, d↑} == 1      : ", ops.d_up * ops.dag_up + ops.dag_up * ops.d_up ≈ I4)
println("{d↓, d↓} == 1      : ", ops.d_dn * ops.dag_dn + ops.dag_dn * ops.d_dn ≈ I4)
println("{d↑, d↑} == 0      : ", ops.d_up * ops.dag_dn + ops.dag_dn * ops.d_up ≈ zeros(DIM, DIM))
println("{d↑, d↓} == 0      : ", ops.d_up * ops.d_dn + ops.d_dn * ops.d_up ≈ zeros(DIM, DIM))
println("d↑² == d↓² == 0     : ", ops.d_up^2 ≈ zeros(DIM, DIM) && ops.d_dn^2 ≈ zeros(DIM, DIM))
println("[H, N] == 0          : ", ops.H * ops.N ≈ ops.N * ops.H)
println("[H, Sz] == 0        : ", ops.H * ops.Sz ≈ ops.Sz * ops.H)

# the fermionic sign itself
up_dn = [0.0, 0.0, 0.0, 1.0]
up     = [0.0, 1.0, 0.0, 0.0]
println("\nd↓|↑⟩ = -|↑⟩      : ", ops.d_dn * up_dn ≈ -up)
println("d↑|↑⟩ = +|↑⟩      : ", ops.d_up * up_dn ≈ [0.0, 0.0, 1.0, 0.0])
```

{d↑, d↑} == 1 : true
{d↓, d↓} == 1 : true
{d↑, d↓} == 0 : true
{d↑, d↓} == 0 : true
d↑² == d↓² == 0 : true
[H, N] == 0 : true
[H, Sz] == 0 : true

d↓|↑⟩ = -|↑⟩ :
true
d↑|↑⟩ = +|↑⟩ : true

In [4]:

```
sp = spectrum(model)

println("eigenenergies E = ", sp.E)
println("closed form      = ", eigenenergies_analytic(model))
println("agree               : ", sp.E ≈ eigenenergies_analytic(model))

println("\nH·V == V·diag(E) : ", ops.H * sp.V ≈ sp.V * Diagonal(sp.E))
println("V orthonormal      : ", permutedims(sp.V) * sp.V ≈ Matrix{Float64}(I, DIM, DIM))
```

eigenenergies E =
[-1.0, -1.0, 0.0, 0.0]
closed form =
[-1.0, -1.0, 0.0, 0.0]
agree : true

H·V == V·diag(E) : true
V orthonormal : true

```
In [5]: # Check Z by hand. With  $u = U/2$  and  $\epsilon d = -U/2$  the spectrum is
#  $E(|0\rangle) = 0$ ,  $E(|\uparrow\rangle) = E(|\downarrow\rangle) = -u$ ,  $E(|\uparrow\downarrow\rangle) = 2\epsilon d + U = 0$ ,
# so  $Z = 1 + e^{\beta u} + e^{\beta u} + 1 = 2(1 + e^{\beta u})$ .
Z_hand = partition_function_analytic(model)

@printf("Z (diagonalisation) = %.15g\n", sp.Z)
@printf("Z (by hand) = %.15g\n", Z_hand)
@printf("relative difference = %.3e\n", abs(sp.Z - Z_hand) / Z_hand)
println("agree to machine precision: ", isapprox(sp.Z, Z_hand; rtol = 1e-12))
```

```
Z (diagonalisation) = 298.826318205153
Z (by hand) = 298.826318205153
relative difference = 1.902e-16
agree to machine precision:
true
```

```
In [6]: # Boltzmann weights and the quantum numbers of each eigenstate.
Nvals, Szvals = quantum_numbers(sp, ops)

println(" i      E_i      p_i      N      Sz")
println("-^50")
for i in 1:DIM
    @printf("%2d %9.5f %11.4e %5.1f %6.1f\n", i, sp.E[i], sp.p[i], Nvals[i], Szvals[i])
end
@printf("\nΣ p_i = %.15g\n", sum(sp.p))
```

i	E_i	p_i	N	Sz
1	-1.00000	4.9665e-01	1.0	-0.5
2	-1.00000	4.9665e-01	1.0	0.5
3	0.00000	3.3464e-03	0.0	0.0
4	0.00000	3.3464e-03	2.0	0.0

```
Σ p_i = 1
```

```
In [7]: # Particle-hole symmetry at  $\epsilon d = -U/2$  pins the atom to half filling.
@printf("(n↑) = %.15g\n", thermal_average(sp, ops.n_up))
@printf("(n↓) = %.15g\n", thermal_average(sp, ops.n_dn))
@printf("(N) = %.15g\n", thermal_average(sp, ops.N))
@printf("(Sz) = %.3e\n", thermal_average(sp, ops.Sz))
@printf("(n↑n↓) = %.15g (exact: 1/Z = %.15g)\n",
    thermal_average(sp, ops.n_up * ops.n_dn, 1 / sp.Z))
@printf("(H) = %.15g\n", thermal_average(sp, ops.H))

# tanh( $\beta u/2$ ) of Eq. (85) is this spectrum's polarisation
u = half_interaction(model)
singly = 2 * exp( $\beta * u$ ) / Z_hand
@printf("\n2·P(singly occupied) - 1 = %.15g\tanh( $\beta u/2$ ) = %.15g\n",
    2 * singly - 1, tanh( $\beta * u / 2$ ))
```

```
(n↑) = 0.5
(n↓) = 0.5
(N) = 1
(Sz) = 0.000e+00
(n↑n↓) = 0.00334642546214243 (exact: 1/Z = 0.00334642546214243)
(H) = -0.993307149075715
```

```
2·P(singly occupied) - 1 = 0.98661429815143
tanh( $\beta u/2$ ) = 0.98661429815143
```

```
In [8]: # What Step 2 will consume: operators in the energy eigenbasis, Eq. (28).
d_rot = to_eigenbasis(sp, ops.d_up)
println("d↑ in the energy eigenbasis:")
display(round(d_rot, digits = 12))
println("\nnonzero elements (i, j, value), each lowering N by one:")
for j in 1:DIM, i in 1:DIM
    if abs(d_rot[i, j]) > 1e-10
        @printf(" (%d, %d) % .6f N: %.1f → %.1f\n",
            i, j, d_rot[i, j], Nvals[j], Nvals[i])
    end
end
```

```
d↑ in the energy eigenbasis:
4x4 Matrix{Float64}:
0.0 0.0 0.0 1.0
0.0 0.0 0.0 0.0
0.0 1.0 0.0 0.0
0.0 0.0 0.0 0.0
nonzero elements (i, j, value), each lowering N by one:
(3, 2) 1.000000 N: 1.0 → 0.0
(1, 4) 1.000000 N: 2.0 → 1.0
```

```
In [9]: # Z against its closed form across a parameter sweep, including  $U = 0$  and  $U < 0$ .
println(" U      β      Z (diag)      Z (hand)      rel.err")
println("-^66")
```

```

for (Ui, βi) in [(1.0, 1.0), (2.0, 5.0), (4.0, 0.3), (0.0, 2.0), (8.0, 10.0), (-2.0, 1.5)]
    mi = HubbardAtomModel(Ui, βi)
    spi = spectrum(mi)
    Zi = partition_function_analytic(mi)
    @printf("%6.1f %6.1f %16.8e %16.8e %10.2e\n",
           Ui, βi, spi.Z, Zi, abs(spi.Z - Zi) / Zi)
end

```

U	β	Z (diag)	Z (hand)	rel.err
1.0	1.0	5.29744254e+00	5.29744254e+00	1.68e-16
2.0	5.0	2.98826318e+02	2.98826318e+02	1.90e-16
4.0	0.3	5.64423760e+00	5.64423760e+00	0.00e+00
0.0	2.0	4.00000000e+00	4.00000000e+00	0.00e+00
8.0	10.0	4.70770534e+17	4.70770534e+17	0.00e+00
-2.0	1.5	2.44626032e+00	2.44626032e+00	0.00e+00

Step 2 — partial spectral functions

Eq. (28), implemented literally and kept general in ℓ , so the one function serves $\ell = 2$ (Step 4), $\ell = 3$ and $\ell = 4$ (Step 5):

$$S[\mathcal{O}_p](\omega'_p) = \sum_{1 \dots \ell} \rho_1 \left[\prod_{i=1}^{\ell-1} (O_{\bar{i}})_{i,i+1} \delta(\omega'_{1 \dots \bar{i}} - E_{i+1,1}) \right] (O_{\bar{\ell}})_{\ell,1},$$

$$E_{i+1,1} = E_{i+1} - E_1.$$

`psf` returns one `PSFTerm` per eigenstate cycle with nonvanishing weight, each carrying

- **position** — the $\ell - 1$ partial sums $\omega'_{1 \dots \bar{i}} = E_{i+1} - E_1$ at which the deltas sit. These are precisely the arguments the Eq. (46) kernel consumes, through $\Omega_{1 \dots \bar{i}} = i\omega_{1 \dots \bar{i}} - \omega'_{1 \dots \bar{i}}$.
- **weight** — $\rho_1 \prod_{i=1}^{\ell} (O_{\bar{i}})_{i,i+1}$, the product taken *cyclically* so the last factor is $(O_{\bar{\ell}})_{\ell,1}$. This is exactly the numerator of Eq. (29), which is what Eqs. (26)–(28) collapse to.

Operators are handed over in the permuted order $\mathcal{O}_p = (O_{\bar{1}}, \dots, O_{\bar{\ell}})$, so one call covers one permutation p ; Steps 4 and 5 loop over permutations and pair each

`position` with the matching $i\omega_{1 \dots \bar{i}}$.

Because the PSFs are sums of δ 's, the ω' integral in Eq. (39) will be a **finite sum** over these terms — no grids, no broadening.

```

In [10]: # ℓ = 2, the simplest case: S[d↑, d↑].
terms2 = psf(sp, ops.d_up, ops.dag_up)
println("raw terms: ", length(terms2), " (one per eigenstate cycle with nonzero weight)\n")
println("      ω'      weight      eigenstate cycle")
println("-^50")
for t in terms2
    @printf("%8.3f %12.6e %s\n", t.position[1], real(t.weight), string(t.states))
end

# Merging coincident deltas gives the PSF as an actual measure.
println("\naggregated:")
for t in aggregate_psf(terms2)
    @printf("%8.3f %12.6e\n", t.position[1], real(t.weight))
end

```

raw terms: 2

(one per eigenstate cycle with nonzero weight)

ω'	weight	eigenstate cycle
-1.000	3.346425e-03	[3, 2]
1.000	4.966536e-01	[1, 4]

aggregated:

-1.000	3.346425e-03
1.000	4.966536e-01

```
In [11]: # Closed form for this PSF. d† connects |↑⟩→|0⟩ and |↑↓⟩→|↓⟩, so the deltas sit
# at ω' = ±u with weights p|0⟩ = 1/Z and p|↓⟩ = e^{βu}/Z.
agg2 = aggregate_psf(terms2)
byw = Dict{round(t.position[1]; digits = 9) => real(t.weight) for t in agg2}

@printf("weight at ω' = -u = %5.2f : %.15g (exact 1/Z = %.15g)\n",
-u, byw[round(-u; digits = 9)], 1 / sp.Z)
@printf("weight at ω' = +u = %5.2f : %.15g (exact e^{βu}/Z = %.15g)\n",
u, byw[round(u; digits = 9)], exp(β * u) / sp.Z)
```

```
weight at ω' = -u = -1.00 : 0.00334642546214243 (exact 1/Z = 0.00334642546214243)
weight at ω' = +u = 1.00 : 0.496653574537858 (exact e^{βu}/Z = 0.496653574537857)
```

```
In [12]: # Sum rule. Summing Eq. (28) over every eigenstate cycle collapses the cyclic
# product to a trace:
# Σ weights = tr(p 0_1~ 0_2~ ... 0_r) = ⟨0_1~ 0_2~ ... 0_r⟩,
# the equal-time expectation value of the operator product in the same order.
# This holds for every ℓ, and is the main check on Step 2.
labelled = [
    ("(d†, d††)", (ops.d_up, ops.dag_up)),
    ("(d††, d†)", (ops.dag_up, ops.d_up)),
    ("(n†, n†)", (ops.n_up, ops.n_dn)),
    ("(n†, d†, d††)", (ops.n_up, ops.d_up, ops.dag_up)),
    ("(d†, n†, d††)", (ops.d_up, ops.n_up, ops.dag_up)),
    ("(d†, d††, d†, d††)", (ops.d_up, ops.dag_up, ops.d_up, ops.dag_up)),
    ("(d†, d††, d↓, d††)", (ops.d_up, ops.dag_up, ops.d_dn, ops.dag_dn)),
    ("(d†, d†↓, d↓, d††)", (ops.d_up, ops.dag_dn, ops.d_dn, ops.dag_up)),
]

println("ℓ operator tuple terms Σ weights (0_1~0_r) |diff|")
println("-"^82)
for (name, Os) in labelled
    terms = psf(sp, Os)
    lhs = real(psf_total_weight(terms))
    rhs = thermal_average(sp, reduce(*, Os))
    @printf(" %d %-22s %5d %14.10f %14.10f %8.1e\n",
        length(Os), name, length(terms), lhs, rhs, abs(lhs - rhs))
end
```

ℓ	operator tuple	terms	Σ weights	(0_1~0_r)	diff
2	(d†, d††)	2	0.5000000000	0.5000000000	0.0e+00
2	(d††, d†)	2	0.5000000000	0.5000000000	0.0e+00
2	(n†, n†)	1	0.0033464255	0.0033464255	0.0e+00
3	(n†, d†, d††)	0	0.0000000000	0.0000000000	0.0e+00
3	(d†, n†, d††)	2	0.5000000000	0.5000000000	0.0e+00
4	(d†, d††, d†, d††)	2	0.5000000000	0.5000000000	0.0e+00
4	(d†, d††, d↓, d††)	1	0.0033464255	0.0033464255	0.0e+00
4	(d†, d†↓, d↓, d††)	1	0.4966535745	0.4966535745	0.0e+00

```
In [13]: # A first look ahead to Step 4. The local spectral function is
# A(ω) = S[d, d†](ω) + S[d†, d](-ω),
# and Appendix E's exact propagator G(iv) = ½ Σ± (iv ± U/2)^{-1} says it must be
# two poles of weight ½ at ω = ±u -- with no temperature dependence at all,
# even though each PSF separately is strongly β dependent.
A = Dict{Float64, Float64}()
for t in aggregate_psf(psf(sp, ops.d_up, ops.dag_up))
    k = round(t.position[1]; digits = 9)
    A[k] = get(A, k, 0.0) + real(t.weight)
end
for t in aggregate_psf(psf(sp, ops.dag_up, ops.d_up))
    k = round(-t.position[1]; digits = 9) # note the reflection ω → -ω
    A[k] = get(A, k, 0.0) + real(t.weight)
end

println("ω A(ω) expected")
println("-"^34)
for k in sort(collect(keys(A)))
    @printf("%8.3f %9.6f %9.6f\n", k, A[k], 0.5)
end
@printf("\nΣ A(ω) = %.15g (sum rule ({d, d†} = 1)\n", sum(values(A)))
```

ω	A(ω)	expected
-1.000	0.500000	0.500000
1.000	0.500000	0.500000

$$\Sigma A(\omega) = 1 \quad (\text{sum rule } \{\{d, d\}\} = 1)$$

```
In [14]: # l = 4: the object Step 5 will need, here for one of the 24 permutations.
0s4 = (ops.d_up, ops.dag_up, ops.d_dn, ops.dag_dn)
terms4 = psf(sp, 0s4)

println("l = 4, permutation (d↑, d↑↑, d↓, d↓↓)")
println("raw terms: ", length(terms4), " distinct positions: ", length(aggregate_psf(terms4)))
println("\n   ω'₁   ω'₁₂   ω'₁₂₃   weight   cycle")
println("-" ^ 62)
for t in terms4
    @printf("%7.3f %7.3f %7.3f %12.6e %s\n",
            t.position[1], t.position[2], t.position[3], real(t.weight), string(t.states))
end

# The individual frequencies behind those partial sums conserve energy.
for t in terms4
    ω = psf_frequencies(t)
    @printf("\nω' = %s\n Σω' = %.2e (energy conservation ω'₁=0)\n",
            string(round.(ω, digits = 6)), sum(ω))
end
```

```
l = 4, permutation (d↑, d↑↑, d↓, d↓↓)
raw terms: 1 distinct positions: 1

   ω'₁   ω'₁₂   ω'₁₂₃   weight   cycle
-----
-1.000   0.000  -1.000  3.346425e-03  [3, 2, 3, 1]
ω' = [-1.0, 1.0, -1.0, 1.0]
Σω' = 0.00e+00 (energy conservation ω'₁=0)
```

Step 3 — the kernel

Eq. (46), the two-branch form, with an explicit test for a vanishing composite $\Omega_{\bar{1}\dots\bar{j}}$:

$$K(\Omega_p) = \begin{cases} \prod_{i=1}^{\ell-1} \Omega_{\bar{1}\dots\bar{i}}^{-1}, & \text{if } \prod_{i=1}^{\ell-1} \Omega_{\bar{1}\dots\bar{i}} \neq 0, \\ -\frac{1}{2} \left[\beta + \sum_{\substack{i=1 \\ i \neq j}}^{\ell-1} \Omega_{\bar{1}\dots\bar{i}}^{-1} \right] \prod_{\substack{i=1 \\ i \neq j}}^{\ell-1} \Omega_{\bar{1}\dots\bar{i}}^{-1}, & \text{if } \Omega_{\bar{1}\dots\bar{j}} = 0, \end{cases}$$

with $\Omega_{\bar{1}\dots\bar{i}} = i\omega_{\bar{1}\dots\bar{i}} - \omega'_{\bar{1}\dots\bar{i}}$. Deliberately **not** Eq. (45) with an ϵ -regulator: Eq. (46) never forms a divergent factor at all — the vanishing one is *excluded* from both the sum and the product — so it is finite by construction.

One design decision. Ω vanishes only when *both* parts do: the Matsubara part $\omega_{\bar{1}\dots\bar{j}}$ and, through a degeneracy $E_{j+1} = E_{\bar{1}}$, the spectral part $\omega'_{\bar{1}\dots\bar{j}}$. The Matsubara part is exactly zero or not at all, so `MatsubaraFreq` stores the **integer** m in $\omega = m\pi/\beta$ and the test is an integer comparison. Float-comparing Ω to zero would confuse a genuinely small frequency — π/β is tiny at large β — with a composite that is identically zero and needs the anomalous branch.

```
In [15]: # Regular branch: a plain product of inverse composites.
Qreg = ComplexF64[1.5 + 0.5im, -2.0 + 1.0im, 0.25 - 3.0im]
println("regular K = ", matsubara_kernel(Qreg, [false, false, false], β))
println(" || Ω⁻¹ = ", prod(inv, Qreg), "\n")

# Anomalous branch, l = 2: empty product and empty sum leave K = -β/2.
println("l=2 anomalous K = ", matsubara_kernel(ComplexF64[0.0], [true], β),
        " (-β/2 = ", -β/2, ") \n")

# Anomalous branch, l = 4 with j = 2. This is literally the second part of
```

```
# Eq. (B26): -1/2[β + 1/Ω_1 + 1/Ω_1^2 3] / (Ω_1^2 Ω_1^2 3).
Ωan = ComplexF64[1.5 + 0.5im, 0.0, -0.75 + 2.0im]
K = matsubara_kernel(Ωan, [false, true, false], β)
B26 = -0.5 * (β + inv(Ωan[1]) + inv(Ωan[3])) * inv(Ωan[1]) * inv(Ωan[3])
println("ℓ=4 anomalous K = ", K)
println("Eq. (B26)      = ", B26)
println("agree         : ", K ≈ B26)
println("finite        : ", isfinite(real(K)) && isfinite(imag(K)),
      " (no 1/0 is ever formed)")
```

```
regular K = 0.005517241379310348
- 0.09379310344827586im
| Ω-1 = 0.005517241379310337 - 0.09379310344827586im
```

```
ℓ=2 anomalous K = -2.5 - 0.0im (-β/2 = -2.5)
```

```
ℓ=4 anomalous K = 0.5797860761869017
+ 0.5660048789641584im
Eq. (B26)      = 0.5797860761869018 + 0.5660048789641583im
agree          : true
finite         : true (no 1/0 is ever formed)
```

```
In [16]: # Why the integer index matters. At β = 1e12 a genuinely nonzero bosonic
# frequency is numerically smaller than any sane float tolerance, yet Ω ≠ 0
# and the regular branch is the correct one.
βbig = 1e12
msbig = [MatsubaraFreq(1), MatsubaraFreq(1), MatsubaraFreq(-1), MatsubaraFreq(-1)]
Ωb, vanb = composites(msbig, [0.0, 0.0, 0.0], βbig; atol = 1e-10)
@printf("Ω12 = %.3e (|Ω12| < atol = 1e-10)\n", abs(Ωb[2]))
println("flagged as vanishing? ", vanb[2], " <- integer test says m12 = ",
      msbig[1].m + msbig[2].m, " ≠ 0")
println("so the kernel takes the regular branch: ",
      matsubara_kernel(Ωb, vanb, βbig) ≈ prod(inv, Ωb))
```

```
Ω12 = 6.283e-12 (|Ω12| < atol = 1e-10)
flagged as vanishing?
false <- integer test says m12 = 2 ≠ 0
so the kernel takes the regular branch: true
```

Step 4 — first milestone, $\ell = 2$

Assemble Eq. (39),

$$G(i\omega) = \sum_p \zeta_p \int d^{\ell-1} \omega'_p K(i\omega_p - \omega'_p) S[\mathcal{O}_p](\omega'_p),$$

for $\mathcal{O} = (d_\sigma, d_\sigma^\dagger)$ and compare against the exact propagator of Appendix E,

$$G(i\nu) = \frac{1}{2} \sum_{\pm} (i\nu \pm U/2)^{-1} = \frac{-i\nu}{\nu^2 + u^2}.$$

The sign ζ_p is $+1$ (-1) if $\mathcal{O}_p$ differs from $\mathcal{O}$ by an even (odd) number of transpositions of **fermionic** operators — so it is not simply the parity of p when bosonic operators are present.

The paper's stop rule: do not proceed until this matches to machine precision.

```
In [17]: println(" n |          G(iv) from Eq. (39)          |          exact (App. E)          | rel err")
println("-" ^ 96)
for n in 0:5
    v = fermionic_freq(n)
    got = propagator(model, v; sp = sp, ops = ops)
    want = propagator_exact(model, v)
    @printf("%3d | %+.15f %+.15fim | %+.10f %+.10fim | %.2e\n",
      n, real(got), imag(got), real(want), imag(want), abs(got-want)/abs(want))
end
```

n	G(iv) from Eq. (39)		exact (App. E)		rel err
0	+0.0000000000000000	-0.450477243368389im	+0.0000000000000000	-0.4504772434im	1.23e-16
1	+0.0000000000000000	-0.413997749252169im	+0.0000000000000000	-0.4139977493im	0.00e+00
2	+0.0000000000000000	-0.289025482222236im	+0.0000000000000000	-0.2890254822im	1.92e-16
3	+0.0000000000000000	-0.216188454372683im	+0.0000000000000000	-0.2161884544im	1.28e-16
4	+0.0000000000000000	-0.171476420146551im	+0.0000000000000000	-0.1714764201im	1.62e-16
5	+0.0000000000000000	-0.141719536856365im	+0.0000000000000000	-0.1417195369im	3.92e-16

In [18]:

```
# The milestone, swept: every parameter set, n = -8 ... 8.
println("      U      β      | max rel err |G_num - G_exact|/|G_exact| over n = -8..8")
println("-^72")
for (Ui, βi) in [(1.0,1.0), (2.0,5.0), (4.0,0.3), (0.0,2.0), (8.0,10.0), (-2.0,1.5)]
    mi = HubbardAtomModel(Ui, βi); spi = spectrum(mi); opsi = operators(mi)
    worst = 0.0
    for n in -8:8
        v = fermionic_freq(n)
        g = propagator(mi, v; sp = spi, ops = opsi); e = propagator_exact(mi, v)
        worst = max(worst, abs(g - e) / abs(e))
    end
    @printf("%6.1f %6.1f      | %.3e\n", Ui, βi, worst)
end
```

U	β	max rel err G_num - G_exact / G_exact over n = -8..8
1.0	1.0	4.364e-16
2.0	5.0	3.917e-16
4.0	0.3	2.545e-16
0.0	2.0	1.962e-16
8.0	10.0	2.314e-16
-2.0	1.5	1.792e-16

Step 5 — $\ell = 4$

$\mathcal{O} = (d_{\sigma}, d_{\sigma}^{\dagger}, d_{\sigma'}, d_{\sigma'}^{\dagger})$ over all 24 permutations. Subtract the disconnected part, Eq. (73),

$$G_{\sigma\sigma'}^{\text{dis}} = \beta G(i\omega_1)G(i\omega_3)(\delta_{\sigma\sigma'}\delta_{\omega_{23},0} - \delta_{\omega_{12},0}),$$

then amputate the external legs, Eq. (74),

$$F_{\sigma\sigma'} = \frac{G_{\sigma\sigma'}^{\text{con}}(i\omega_1, i\omega_2, i\omega_3)}{G(i\omega_1)G(-i\omega_2)G(i\omega_3)G(-i\omega_4)}, \quad \omega_4 = -\omega_{123}.$$

Appendix D does **not** enter here: its imaginary frequency shifts are needed only for the Keldysh amputation of Eq. (76), where the legs sit at real frequencies. On the Matsubara axis the legs are exactly on discrete frequencies and there is nothing to shift.

Eq. (85) and the PDF text layer

The published Eq. (85) is correct. What is broken is the PDF *text layer*: it puts both products of the anomalous term on one line and loses the fraction bar. Extracted literally it reads

$$\beta u^2 \frac{\delta_{\omega_{12}} \text{th} + \delta_{\omega_{13}} (\text{th} - 1) + \delta_{\omega_{14}} (\text{th} + 1)}{\prod_i (i\omega_i + u) \prod_i (i\omega_i)}$$

(dimension energy⁻⁷: cannot be right)

since βu^2 is an energy, so the remaining factor must be dimensionless while that denominator carries energy⁸. Building against that form made Step 5 fail *only* in the $\beta\delta_\omega$ terms, while matching to 10^{-16} at every generic frequency point.

The printed factor is $\prod_i(i\omega_i + u)/\prod_i(i\omega_i)$:

$$F_{\uparrow\downarrow} = 2u + u^3 \frac{\sum_i (i\omega_i)^2}{\prod_i (i\omega_i)} - \frac{6u^5}{\prod_i (i\omega_i)} + \beta u^2 [\delta_{\omega_{12}} \text{th} + \delta_{\omega_{13}} (\text{th} - 1) + \delta_{\omega_{14}} (\text{th} + 1)] \frac{\prod_i (i\omega_i + u)}{\prod_i (i\omega_i)},$$

$$F_{\uparrow\uparrow} = \beta u^2 (\delta_{\omega_{14}} - \delta_{\omega_{12}}) \frac{\prod_i (i\omega_i + u)}{\prod_i (i\omega_i)},$$

with $u = U/2$, $\text{th} = \tanh(\beta u/2)$. This was first reconstructed from three independent arguments — dimensional consistency; Appendix E's separately stated second-order expansion $F_{\uparrow\downarrow} = U + \frac{1}{4}\beta U^2(\delta_{\omega_{14}} - \delta_{\omega_{13}})$, $F_{\uparrow\uparrow} = \frac{1}{4}\beta U^2(\delta_{\omega_{14}} - \delta_{\omega_{12}})$; and a brute-force τ -integration of Eq. (71) that reproduces the computed $G^{(4)}$ independently of this whole machinery — and then **confirmed by rendering p. 16 to an image and reading the printed equation**. Eqs. (28), (39), (45), (46), (73), (74) and the Appendix E results were checked the same way; only Eq. (85) was affected.

Note that whenever any δ fires the frequencies are $\pm$ -paired, $(\nu, -\nu, \nu', -\nu')$, so $\prod_i(i\omega_i + u) = \prod_i(i\omega_i - u)$ and the sign of u in that product is not observable.

```
In [19]: # Both channels at a handful of points: generic ones, and ones where a  $\delta$  fires.
println(" n1 n2 n3 | 612 613 614 | F $\uparrow\downarrow$  computed F $\uparrow\downarrow$  exact | F $\uparrow\uparrow$  computed F $\uparrow\uparrow$  exact")
println("-^104")
for (n1, n2, n3) in [(0,0,0), (3,0,-2), (1,-1,0), (1,-2,0), (0,0,-1), (0,-1,0)]
    ms = vertex_frequencies(n1, n2, n3)
    d(k) = ms[1].m + ms[k].m == 0 ? 1 : 0
    fud, eud = vertex(model, :up, :dn, ms; sp=sp, ops=ops), vertex_exact(model, :updn, ms)
    fuu, euu = vertex(model, :up, :up, ms; sp=sp, ops=ops), vertex_exact(model, :upup, ms)
    @printf("%3d%3d%3d | %3d%4d%4d | %13.6f %13.6f | %13.6f %13.6f\n",
           n1,n2,n3, d(2),d(3),d(4), real(fud),real(eud), real(fuu),real(euu))
end
```

n1	n2	n3	612	613	614	F $\uparrow\downarrow$ computed	F $\uparrow\downarrow$ exact	F $\uparrow\uparrow$ computed	F $\uparrow\uparrow$ exact
0	0	0	0	0	0	24.964596	24.964596	-0.000000	-0.000000
3	0	-2	0	0	0	-0.393066	-0.393066	0.000000	0.000000
1	-1	0	0	0	1	37.064469	37.064469	22.636964	22.636964
1	-2	0	1	0	0	14.427505	14.427505	-22.636964	-22.636964
0	0	-1	0	1	1	76.522586	76.522586	62.411490	62.411490
0	-1	0	1	0	1	138.934076	138.934076	0.000000	0.000000

```
In [20]: # Swept over n1,n2,n3  $\in$  -3:3 (343 points). Where F is identically zero we
# require it numerically, measured against the scale of F on the grid.
println("      U       $\beta$  | chan | max rel err (|F|>0) | max |F_num| where F  $\equiv$  0 | scale of F")
println("-^90")
for (Ui,  $\beta$ i) in [(1.0,1.0), (2.0,5.0), (4.0,0.3), (8.0,10.0), (-2.0,1.5)]
    mi = HubbardAtomModel(Ui,  $\beta$ i); spi = spectrum(mi); opsi = operators(mi)
    for (sa, sb, tag) in [(:up,:dn,:updn), (:up,:up,:upup)]
        re = 0.0; ze = 0.0; scale = 0.0
        for n1 in -3:3, n2 in -3:3, n3 in -3:3
            ms = vertex_frequencies(n1, n2, n3)
            got = vertex(mi, sa, sb, ms; sp=spi, ops=opsi)
            want = vertex_exact(mi, tag, ms)
            scale = max(scale, abs(want))
            abs(want) > 1e-6 ? (re = max(re, abs(got-want)/abs(want))) :
```

```

        (ze = max(ze, abs(got)))
    end
    @printf("%6.1f %6.1f | %4s |      %.3e      |      %.3e      | %.3e\n",
        Ui, βi, tag, re, ze, scale)
end
end

```

U	β	chan	max rel err (F >0)	max F_num where F ≡ 0	scale of F
1.0	1.0	updn	1.238e-13	0.000e+00	1.371e+00
1.0	1.0	upup	7.981e-13	1.115e-14	2.628e-01
2.0	5.0	updn	5.606e-14	0.000e+00	1.389e+02
2.0	5.0	upup	4.764e-15	3.911e-15	6.241e+01
4.0	0.3	updn	5.238e-14	0.000e+00	5.893e+00
4.0	0.3	upup	3.397e-13	7.888e-14	1.289e+00
8.0	10.0	updn	8.721e-16	0.000e+00	1.214e+07
8.0	10.0	upup	5.038e-16	1.673e-11	4.257e+06
-2.0	1.5	updn	2.131e-14	0.000e+00	5.911e+00
-2.0	1.5	upup	4.312e-14	5.746e-15	2.262e+00

Step 6 — null test

Set $U = 0$. Every connected object must vanish identically. This exercises all 24 permutations and every sign ζ_p against a target of exactly zero.

It also drives the anomalous branch of Eq. (46) hard: at $U = 0$ the whole spectrum is degenerate, so $\omega' = 0$ for every PSF term and every vanishing bosonic composite triggers the second branch. A sign error there would show up here.

```

In [21]: println(" β | max |G^con| | max |F| | max |G^dis| (nonzero!)")
println("-^76")
for βi in (0.5, 2.0, 7.0)
    m0 = HubbardAtomModel(0.0, βi); sp0 = spectrum(m0); ops0 = operators(m0)
    gc = 0.0; fv = 0.0; gd = 0.0
    for n1 in -3:3, n2 in -3:3, n3 in -3:3, ch in ([:up,:dn], [:up,:up])
        ms = vertex_frequencies(n1, n2, n3)
        gc = max(gc, abs(connected_4p(m0, ch[1], ch[2], ms; sp=sp0, ops=ops0)))
        fv = max(fv, abs(vertex(m0, ch[1], ch[2], ms; sp=sp0, ops=ops0)))
        gd = max(gd, abs(disconnected_4p(m0, ch[1], ch[2], ms; sp=sp0, ops=ops0)))
    end
    @printf("%5.1f | %.3e | %.3e | %.3e\n", βi, gc, fv, gd)
end
println("\nG^dis is large, so G^con = G - G^dis vanishing is a real cancellation,")
println("not a case of both sides being zero.")

```

β	max G^con	max F	max G^dis (nonzero!)
0.5	4.337e-19	2.737e-14	1.267e-02
2.0	2.776e-17	6.844e-15	8.106e-01
7.0	1.807e-15	1.656e-14	3.475e+01

G^dis is large, so G^con = G - G^dis vanishing is a real cancellation,
not a case of both sides being zero.